

Stochastic simulation of football scores

1. Objective

The objective of this project is to model and simulate the results of 2011/2012 English Football Premier League based on the goals scored that season using stochastic models like Poisson Distribution and Monte Carlo Method.

The Poisson Distribution will treat the goals scored as a independent and a rare event occurring in a fixed amount of time, in this project case, the 90 minutes of a football game. It transforms continuous and abstract variables, such as team average attack and defense strength into discrete probabilities, allowing the model to determine the likelihood of a specific scoreline rather than vague averages.

The Monte Carlo Method is designed to avoid the unpredictability of a single probabilistic simulation by generating parallel iterations of alternative season outcomes.

2. Data set

The data set used for this project is called “engsoccerdata” R package, which provides historical data for several seasons tiers, and competitions of the English Football Premier League from 1871-2015.

In order to create the simulations for this project, it been used the “england” library which contain over 190.000 rows of the following variables:

Date – Date of match;

Season – Season of match (the starting year);

Home – Home team;

Visitor – Visiting team;

FT – Full Time result;

Hgoal – Goals scored by home team;

Vgoal - Goals scored by visiting team;

Division – The football league name until 1992;

Tier – The football league name after 1992;

Totgoal – Total goals scored in game;

Goaldif – Goal difference in games;

Result – Home win, Away win or draw.

3. Methodology

The library “engsoccerdata” was loaded with the specific data set “england” (Fig. 1).

```
library(engsoccerdata)  
data(england)
```

(Fig. 1 “Library and data set load”)

First of all, the data set was filtered to year 2011 which means the season 2011/2012 and the 1st tier league (Fig. 2). It have also been checked if there are all the games registered for this season (Each of the 20 teams plays 38 games. The total is 760, but because every match involves 2 teams, the total have been divided by 2, so there should be 380 games) (Fig. 3).

```
#Selecting a specific season and league (tier)  
real_season <- subset(england, Season == 2011 & tier == 1)  
  
#Checking if there are all the games from that season and tier  
print(paste("Number of games:", nrow(real_season)))
```

(Fig. 2 “Filters”)

```
> #Checking if there are all the games from that season and tier  
> print(paste("Number of games:", nrow(real_season)))  
[1] "Number of games: 380"
```

(Fig. 3 “Number of games”)

Because the project objective is to simulate the goals scored in this season, total goals and goals by team, it have been first calculated the average goals scored and conceded by the home and away teams in order to determine how powerful is every team attack and defense (Fig. 4).

```
#Average goals scored by home and away teams  
team_home_stats <- aggregate(hgoal ~ home, data = real_season, FUN = mean)  
team_away_stats <- aggregate(vgoal ~ visitor, data = real_season, FUN = mean)  
  
#Average conceded goals by home and away teams  
team_home_defense <- aggregate(vgoal ~ home, data = real_season, FUN = mean)  
team_away_defense <- aggregate(hgoal ~ visitor, data = real_season, FUN = mean)
```

(Fig. 4 “Home and away average attack and defense”)

In order to create the simulation, the “template” of the real result table have been saved in a variable named “simulated_season” and two other columns have been created for the prediction of home and away goals scored (Fig. 5)

```
simulated_season <- real_season  
  
simulated_season$Sim_Home_Goal <- NA  
simulated_season$Sim_Away_Goal <- NA
```

(Fig. 5 “New columns in “simulation” table”)

The creation of variables where the total number of expected home and away goals will be stored (Fig. 6).

```
raw_expected_h <- 0  
raw_expected_v <- 0
```

(Fig. 6 “Expected goals variables”)

Because not every time the best teams wins, it have been added a “Form” factor for the code to randomize the scores. If there haven’t been this factor, the “good” teams, would win every time, and especially at a very big difference against “small” or weaker teams (Fig. 7).

```
#Adding a "Form" factor because not the best teams win every time  
simulated_season$Home_Form <- NA  
simulated_season$Away_Form <- NA
```

(Fig. 7 “The Form variable”)

After the raw expected goals and the form factor variables have been created, the calibration for the amount of goals was calculated in order to avoid some big discrepancies between the “better” and the “weaker” team. For a better result it been used the geometrical mean (Fig. 8).

```
#Creating a calibration factor in order to not have some exaggerated results  
for(i in 1:nrow(real_season)) {  
  h_team <- real_season$home[i]  
  a_team <- real_season$visitor[i]  
  
  h_att <- team_home_stats$hgoal[team_home_stats$home == h_team]  
  a_def <- team_away_defense$hgoal[team_away_defense$visitor == a_team]  
  a_att <- team_away_stats$vgoal[team_away_stats$visitor == a_team]  
  h_def <- team_home_defense$vgoal[team_home_defense$home == h_team]  
  
  raw_expected_h <- raw_expected_h + sqrt(h_att * a_def)  
  raw_expected_v <- raw_expected_v + sqrt(a_att * h_def)  
}  
  
total_expected <- raw_expected_h + raw_expected_v  
total_actual <- sum(real_season$hgoal) + sum(real_season$vgoal)
```

(Fig. 8 “The calibration factor”)

In order to align the stochastic model output with the real data it have been created a calibration factor that serves as a correction mechanism to ensure that the total number of simulated goals across the entire season it will match the real number of goals (Fig. 9).

```
calibration_factor <- total_actual / total_expected  
print(paste("Calibration Factor:", round(calibration_factor, 4)))
```

(Fig. 9 “Calculation of the calibration factor”)

In order for the algorithm to find the best solution to match the real scores, it was implemented the Monte Carlo method to run the code 5.000 times in parallel to find the best seed that can go near the real scores (Fig. 10).

```
#Finding the best seed for the prediction
number_of_attempts <- 5000
lowest_error <- Inf
best_season <- NULL
best_seed <- 0
```

(Fig. 10 “5.000 iterations variables”)

It was created a loop that will try seeds from 1 to 5.000 in order to find the best result. It takes every seed and simulates the results of the games to be able to match as closely as possible the real number of goals but also the games results. There applies the Form variable too, that is set to a 5% chance that a team will do better or worst than it's usual (Fig. 11).

```
for(k in 1:number_of_attempts) {
  set.seed(k)
  temp_season <- simulated_season

  #Loop through every game of the season and simulate a score
  for(i in 1:nrow(simulated_season)) {

    current_home_team <- simulated_season$home[i]
    current_away_team <- simulated_season$visitor[i]

    h_attack <- team_home_stats$hgoal[team_home_stats$home == current_home_team]
    a_attack <- team_away_stats$vggoal[team_away_stats$visitor == current_away_team]

    h_defense <- team_home_defense$vggoal[team_home_defense$home == current_home_team]
    a_defense <- team_away_defense$hgoal[team_away_defense$visitor == current_away_team]

    #Implementing a 5% chance that a team can perform better or worst than usual
    home_form <- rnorm(1, mean = 1, sd = 0.05)
    away_form <- rnorm(1, mean = 1, sd = 0.05)

    base_h <- sqrt(h_attack * a_defense)
    day_lambda_home <- max(0.1, base_h * (1 + (home_form - 1) - (away_form - 1)) * calibration_factor)
    base_v <- sqrt(a_attack * h_defense)
    day_lambda_away <- max(0.1, base_v * (1 + (away_form - 1) - (home_form - 1)) * calibration_factor)

    temp_season$Home_Form[i] <- round(home_form, 2)
    temp_season$Away_Form[i] <- round(away_form, 2)

    #Simulating goals using the specific lambdas for both sides
    temp_season$Sim_Home_Goal[i] <- rpois(n = 1, lambda = day_lambda_home)
    temp_season$Sim_Away_Goal[i] <- rpois(n = 1, lambda = day_lambda_away)
  }
}
```

(Fig. 11 “Looping the seeds”)

To show the best result, the algorithm has to compare all the calculations and after that, to save the “best season” in the variable created at the beginning, “simulated season” (Fig. 12).

```
#Comparing the results from the chosen number of simulations
diff_h <- abs(temp_season$Sim_Home_Goal - real_season$hgoal)
diff_v <- abs(temp_season$Sim_Away_Goal - real_season$vgoal)
total_error <- sum(diff_h) + sum(diff_v)

if(total_error < lowest_error) {
  lowest_error <- total_error
  best_season <- temp_season
  best_seed <- k
}
}

simulated_season <- best_season
```

(Fig. 12 “Comparing the simulations”)

The best simulation that the algorithm was able to find was the 4.929th simulation, so near the end of the iterations (Fig. 13).

```
> print(paste("Best simulation found at seed:", best_seed))
[1] "Best simulation found at seed: 4929"
```

(Fig. 13 “Best seed found”)

The first simulation of this project is the comparison between the game results (Fig 14, Fig. 15).

```
#Comparison between simulated and real home and away goals
head(simulated_season[, c("home", "visitor", "hgoal", "vgoal", "Sim_Home_Goal", "Sim_Away_Goal")])
```

(Fig. 14 “Game results comparison code”)

	home	visitor	hgoal	vgoal	Sim_Home_Goal	Sim_Away_Goal
181861	Blackburn Rovers	wolverhampton wanderers	1	2	3	3
181862	Fulham	Aston villa	0	0	1	2
181863	Liverpool	Sunderland	1	1	2	1
181864	Newcastle United	Arsenal	0	0	2	3
181865	Queens Park Rangers	Bolton wanderers	0	4	2	0
181866	wigan Athletic	Norwich City	1	1	0	0

(Fig. 15 “Game results comparison result”)

While the algorithm was a bit far from reality for the first games, managing to almost predict some outcomes, it was able to predict the total sum of the goals that will be scored in that season as it is presented in the second simulation (Fig. 16, Fig. 17).

```
#Comparison between simulated and real total goals
total_real <- sum(real_season$hgoal) + sum(real_season$vgoal)
total_sim <- sum(simulated_season$Sim_Home_Goal) + sum(simulated_season$Sim_Away_Goal)
```

(Fig. 16 “Total goals comparison code”)

```
> print(paste("Total Real Goals:", total_real))  
[1] "Total Real Goals: 1066"  
> print(paste("Total Simulated Goals:", total_sim))  
[1] "Total Simulated Goals: 1039"
```

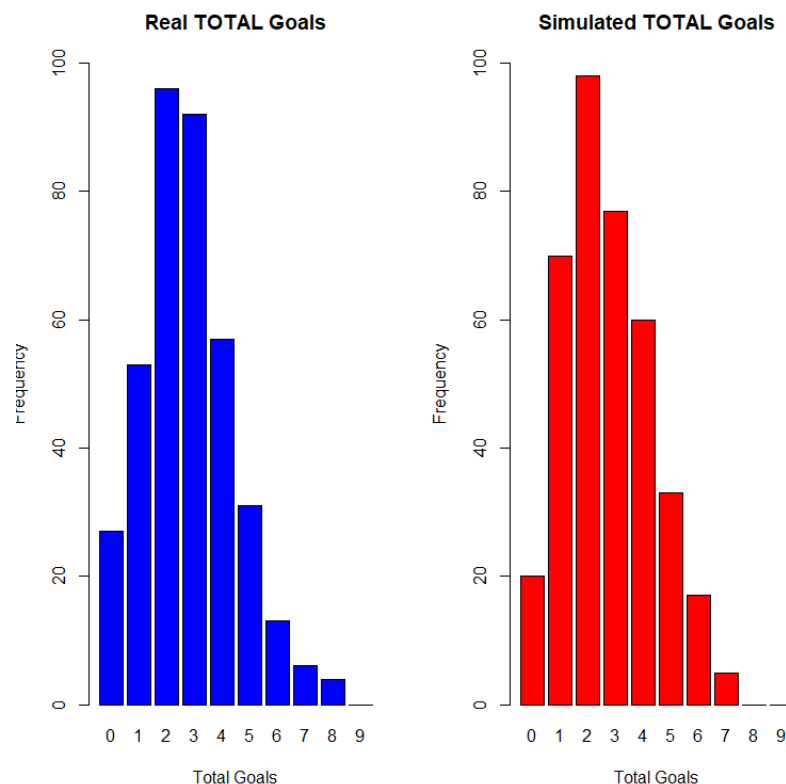
(Fig. 17 "Total goals comparison result")

The simulated goals have a difference of only 27, so the algorithm did a good job in predicting the amount of goals that have been scored in that season.

In order to observe the total goals per game calculated by the algorithm it was created 2 plots for this comparison (Fig. 18, Fig. 19).

```
par(mfrow=c(1,2))  
  
#Real data  
real_counts <- table(factor(real_season$Real_Total_Goals, levels = 0:9))  
barplot(real_counts, main="Real TOTAL Goals",  
        col="blue", xlab="Total Goals", ylab="Frequency", ylim=c(0,100))  
  
# Simulated Total Goals  
sim_counts <- table(factor(simulated_season$Sim_Total_Goals, levels = 0:9))  
barplot(sim_counts, main="Simulated TOTAL Goals",  
        col="red", xlab="Total Goals", ylab="Frequency", ylim=c(0,100))
```

(Fig. 18 "Real and simulated total goals comparison code")



(Fig. 19 "Real and simulated total goals comparison plots")

Some results are totally different, some results are near and some results are good. The total goals being a sum of goals from the “how good” is every team as the algorithm understood by the calculations that have been made.

The closest to reality are the matches that concluded with 2, 4, 5, and 7 goals.

The last simulation in this project is the place in the league table after all the games were played with the real life football rules (Fig. 20, Fig. 21, Fig. 22).

```
#Creating the final tables
get_table <- function(df, h_goal_col, v_goal_col) {
  teams <- unique(c(df$home, df$visitor))
  standings <- data.frame(Team = teams, Points = 0, GD = 0)

  for(i in 1:nrow(df)) {
    h_team <- df$home[i]
    v_team <- df$visitor[i]
    h_score <- df[[h_goal_col]][i]
    v_score <- df[[v_goal_col]][i]

    #Goal difference
    standings$GD[standings$Team == h_team] <- standings$GD[standings$Team == h_team] + (h_score - v_score)
    standings$GD[standings$Team == v_team] <- standings$GD[standings$Team == v_team] + (v_score - h_score)

    #Points
    if(h_score > v_score) {
      standings$Points[standings$Team == h_team] <- standings$Points[standings$Team == h_team] + 3
    } else if(h_score == v_score) {
      standings$Points[standings$Team == h_team] <- standings$Points[standings$Team == h_team] + 1
      standings$Points[standings$Team == v_team] <- standings$Points[standings$Team == v_team] + 1
    } else {
      standings$Points[standings$Team == v_team] <- standings$Points[standings$Team == v_team] + 3
    }
  }
  return(standings)
}
```

(Fig. 20 “Applying the real life football rules to the table”)

```
#Generate Both Tables
real_table <- get_table(real_season, "hgoal", "vgoal")
sim_table <- get_table(simulated_season, "Sim_Home_Goal", "Sim_Away_Goal")

#Merging the tables for comparison
comparison <- merge(real_table, sim_table, by="Team", suffixes = c("_Real", "_Sim"))

#Sort by real points to make the graph look organized
comparison <- comparison[order(comparison$Points_Real),]

print("LEAGUE TABLE COMPARISON")
print(head(comparison))

par(mfrow=c(1,1))
par(mar=c(5, 8, 4, 2))

plot(x = c(min(comparison$Points_Real, comparison$Points_Sim) - 5,
           max(comparison$Points_Real, comparison$Points_Sim) + 5),
     y = c(1, nrow(comparison)),
     type = "n",
     yaxt = "n", xlab = "Points", ylab = "",
     main = "Projected vs Real season points")

abline(h = 1:nrow(comparison), col = "lightgray", lty = "dotted")

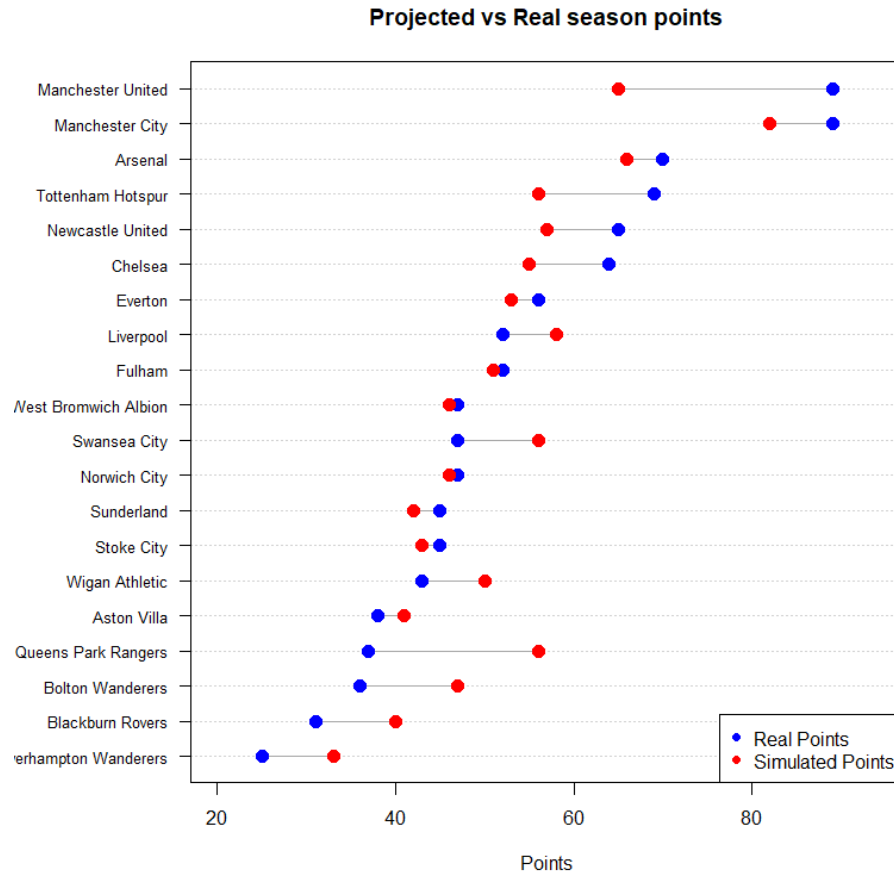
segments(x0 = comparison$Points_Real, y0 = 1:nrow(comparison),
         x1 = comparison$Points_Sim, y1 = 1:nrow(comparison),
         col = "darkgray", lwd = 1.5)

points(comparison$Points_Real, 1:nrow(comparison), pch = 19, col = "blue", cex = 1.5)
points(comparison$Points_Sim, 1:nrow(comparison), pch = 19, col = "red", cex = 1.5)

axis(2, at = 1:nrow(comparison), labels = comparison$Team, las = 2, cex.axis = 0.8)

legend("bottomright", legend=c("Real Points", "Simulated Points"),
      col=c("blue", "red"), pch=19, bg="white")
```

(Fig. 21 “Creating the comparison tables”)



(Fig. 22 “Comparison tables”)

In this case exists some big discrepancies too because the games are won by the number of goals scored. In reality, Manchester United won the title because they won a lot of 1-0 games. Manchester City was on the 2nd place because even if they won a lot of games at a bigger difference, they also conceded many more goals and lost the title at goals difference.

The algorithm simulated that case (and the others where is the case too) because it didn't took in account the tactics of the teams, but the goals scored, which are the subject of this project, so in the algorithm eyes Man. City is better placed than Man. United because they score many more goals than Man. United, so the algorithm considered them more powerful.

With those things taken in consideration, it was able to predict some points very near to the real life result (Fulham, W.B.A., Norwich City, Sunderland, Stoke City and Aston Villa).

4. Conclusion

Using the Poisson Distribution on the data that already exists in the data set, the code managed to simulate the total number of goals that have been scored in the mentioned season, to predict the game results for the entire season and the final league table.

Using Monte Carlo method, the code ran 5.000 iterations of the season in order to identify the simulation that has the lowest difference of the results compared to the real season.

In conclusion, with all the limitations that the project had, the algorithm did a good job in simulating some results and especially the total goals that have been scored in 2011-2012 English Football Premier League.

The discrepancy between the rest of the values comes from real life thinking and tactics. The algorithm understands that every game is played at the same intensity from the 1st minute until the last one, not taking in consideration other tactics or other factors as the fatigue, weather, how good the pitch is or what key players are missing.

It is hard for the algorithm to know which are the teams that will “park the buss” against other teams and won’t be so many goals in that game.

For an improvement of the algorithm to be able to simulate the results even better, there should be added some additional variables for tactics and other random factors as mentioned above, but this also can be a hard challenge because there are a lot of tactics that change during the games based of every match.